
FlexDraft: Flexible Speculative Decoding via Attention Tuning and Bonus-Guided Calibration

Yaojie Zhang^{1,3*} Jianuo Huang^{1,2*} Junlong Ke⁴ Yuhang Han⁵
Yongji Long² Tianchen Zhao⁴ Biqing Qi⁶ Linfeng Zhang^{1†}
¹EPIC Lab, SJTU ²UESTC ³School of Software Engineering, HUST
⁴Tsinghua University ⁵HKUST(GZ) ⁶Shanghai AI Laboratory
{yaojiezhong288, jianuohuang82 }@gmail.com

Abstract

Speculative decoding accelerates memory-bound LLM inference without quality degradation by using a fast drafter to propose multiple candidate tokens and the target model to verify them in parallel. However, conventional sequential speculative decoding suffers from mutual waiting between drafting and verification, and repeated exchange of intermediate states further increases memory access overhead. Parallel speculative decoding addresses this limitation by performing drafting and verification within a single target forward pass, allowing future drafts to be prepared while current candidates are being verified. Although effective at small batch sizes, existing parallel speculative decoding methods either require costly continual pretraining with quality degradation or suffer from low acceptance rates. More importantly, this paradigm inherently suffers from uncertainty in both the bonus token and the accepted length, leading to draft verification mismatch and causing throughput gains to collapse at large batch sizes. To address these limitations, we introduce FlexDraft, a lossless speculative decoding framework that flexibly adapts to varying batch sizes through three key designs. (1) Attention Tuning enables block diffusion drafting by tuning only the attention projectors of the final few layers on mask tokens, while keeping the autoregressive path frozen to exactly preserve the target distribution and produce high quality drafts with minimal trainable parameters. (2) Bonus-guided Calibration uses a lightweight MLP conditioned on the resolved bonus token to calibrate draft logits, mitigating the draft verification mismatch caused by bonus token uncertainty. (3) Flex Decoding dynamically switches between parallel draft and verify at small batch sizes and sequential draft then verify at large batch sizes, and further adjusts verification length based on draft confidence to eliminate redundant computation. Experiments on Qwen3-8B show that FlexDraft achieves an average $4.59\times$ speedup over autoregressive decoding without quality loss. *Our code will be released on Github.*

1 Introduction

Autoregressive large language models (LLMs) have demonstrated remarkable capabilities across diverse natural language understanding and generation tasks [30, 9, 29], but their token by token decoding paradigm makes inference highly memory bound and fundamentally limits generation speed. Speculative decoding addresses this bottleneck as a lossless acceleration paradigm by using a lightweight drafter to propose multiple candidate tokens and the target model to verify them in a single forward pass, thereby amortizing expensive target model computation over multiple accepted tokens [5, 13]. Prior methods improve speculative decoding by enhancing draft quality [37, 17], reducing draft generation cost [1, 6], or organizing candidates more effectively for verification [24, 15], with

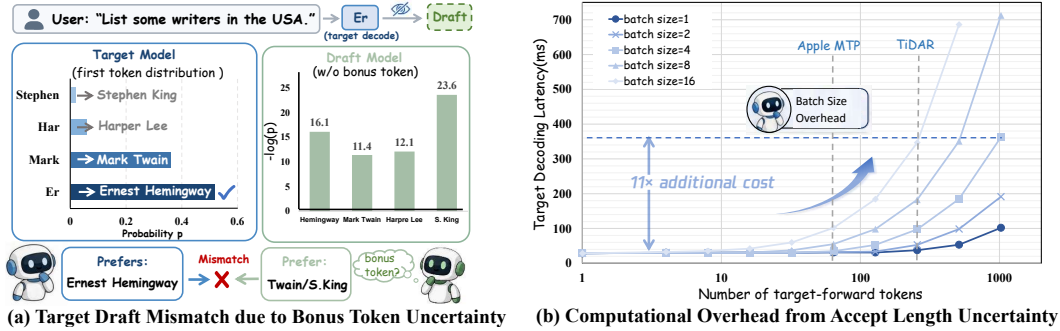


Figure 1: **Limitations of the parallel speculative decoding paradigm.** (a) Given the prefix, the target model predicts the next token and prefers “Er”, the first token of “Ernest Hemingway”, which constrains the subsequent generation toward Hemingway. Without access to the bonus token, the draft model tends to favor alternative continuations. (b) Accept length uncertainty forces parallel speculative decoding to consider all potentially accepted prefixes of the last draft, requiring mask embeddings to traverse all target layers and causing the target forward token count to grow as $O(N^2)$. The right figure analyzes the resulting overhead for different numbers of target forward tokens, which is moderate at small batch sizes but grows rapidly at larger batch sizes.

representative examples such as EAGLE for feature-level draft prediction [16, 17] and DFlash for target conditioned block diffusion drafting [6].

Despite these advances, most conventional speculative decoding methods still follow a sequential draft then verify schedule, where the next draft cannot begin until verification completes. This serialized execution induces mutual waiting between the drafter and the target model, and repeated exchange of intermediate states further increases memory access overhead, limiting end-to-end speedup [22, 28].

To reduce this mutual waiting, recent methods prepare future candidates before verification is fully resolved, enabling parallel speculative decoding that overlaps drafting with verification within a single target forward pass [19, 21, 27]. However, existing parallel speculative decoding methods either require continual pretraining with substantial training and parameter overhead, potentially leading to quality degradation, or suffer from low acceptance rates. More importantly, this paradigm introduces two fundamental limitations. Since future drafts are generated before the verification outcome is known, the drafter faces uncertainty in both the bonus token and the accepted length. These two uncertainties affect parallel future drafting in different ways. Bonus token uncertainty weakens draft verification alignment, while acceptance length uncertainty forces the drafter to generate candidate branches for multiple possible accepted prefixes, introducing substantial redundant drafting overhead. We detail these two limitations below.

1 Bonus token uncertainty. In speculative decoding, when the first rejection occurs during verification, the target model samples a corrected token at the rejected position, which is commonly referred to as the *bonus token*. However, in parallel speculative decoding, this bonus token is not available when the draft tokens are generated [19, 21]. As a result, the draft model must predict future tokens without knowing which corrected token will be produced by the target model, leading to a mismatch between drafting and verification. Figure 1(a) illustrates this issue. Given the query “List some writers in the USA.”, the target model may produce Er as the bonus token, corresponding to the beginning of Ernest Hemingway. Conditioned on this token, the subsequent verified trajectory should remain coherent with Ernest Hemingway. In contrast, because the draft model cannot observe the bonus token during drafting, it may assign high probability to continuations associated with other writers, such as Mark Twain or S. King. Although these drafts may be locally plausible under the original prefix, they become inconsistent once the verifier selects Er as the bonus token. This mismatch reduces the probability that the drafted tokens can be accepted, thereby limiting the speedup of parallel speculative decoding.

2 Acceptance length uncertainty. In parallel draft and verify methods, future candidates are generated before the actual acceptance length is known [21, 27, 19]. Since the verifier determines the accepted length only after verification, the drafter cannot know which future branch will remain valid and must prepare candidates for multiple possible accepted prefixes, introducing substantial redundant computation. In existing parallel speculative decoding methods, the additional mask tokens

used for future drafting are injected from the bottom layers and must pass through all target layers together with the verification tokens. Consequently, as the draft block length increases, covering all possible accepted prefixes requires a rapidly growing number of mask tokens, leading to an $O(N^2)$ increase in the effective number of target forward tokens. This creates an inherent trade-off. Methods with low acceptance rates usually adopt short draft blocks, because larger blocks bring limited accepted token gains but much higher mask token overhead. Methods such as TiDAR can obtain longer accepted continuations through continual pretraining, but their larger blocks further amplify the redundant target forward computation caused by acceptance length uncertainty. As shown in Figure 1(b), the target side overhead exceeds $11\times$ at batch size 16. Consequently, the latency savings from overlapping drafting and verification are ultimately offset or even outweighed by the extra target forward computation, causing speedup to collapse at large batch sizes.

To address these challenges, we introduce FlexDraft, a speculative decoding framework that flexibly adapts to varying batch sizes without quality degradation, built on three key contributions. **(1) Attention Tuning (Attn Tuning)** tunes only the attention projectors of the final few layers for mask token prediction, while keeping the autoregressive verification path frozen. It adds trainable parameters equivalent to only about **6%** of the target model size, avoiding full model adaptation while preserving the target model’s original behavior. **(2) Bonus-guided Calibration** conditions on the bonus token embedding and the hidden states of mask tokens to produce a calibration bias over the draft logits. This mechanism guides the draft distribution to be consistent with the determined bonus token, improving alignment between drafting and verification. **(3) Flex Decoding** dynamically switches between parallel draft and verify at small batch sizes and sequential draft then verify at large batch sizes, while adaptively pruning the verification length based on draft confidence to eliminate redundant computation.

2 Related Work

Speculative Decoding Speculative decoding accelerates LLM inference by decomposing generation into a fast draft phase and a lossless verification phase, where a lightweight drafter proposes multiple candidate tokens and the target model verifies them in parallel using the standard speculative sampling rule [5, 13]. Existing methods improve speculative decoding mainly by enhancing draft quality, reducing draft generation cost, or organizing candidates more efficiently for verification [35]. To improve draft quality and acceptance rates, prior work has strengthened draft target alignment and draft prediction through distillation [37], head-based speculation [4, 2, 36, 18]. Representative methods include Medusa style approaches, which attach trainable decoding heads to the target model [4], and the EAGLE series, which improves drafting with target hidden states, multi-layer feature fusion, and direct token prediction [16, 17]. To reduce drafter cost or latency, retrieval-based methods [11, 10, 26] avoid a separately trained draft model by constructing lightweight draft sources from external datastores. Parallel drafting methods further reduce latency by replacing sequential autoregressive drafting with parallel proposal generation [8, 14, 6, 20]. On the verification side, tree-structured speculation methods organize candidate continuations into token trees for parallel verification [24, 15]. Despite these advances, most methods still follow the conventional draft then verify schedule, where the next draft cannot start until verification finishes, causing mutual waiting and limiting end-to-end speedup [22]. Recent methods attempt to alleviate this bottleneck by preparing future candidates before verification is fully resolved. BiTA [19] performs draft generation and verification in parallel within a single enhanced model, and MTP-based methods [27] speculate over multiple future positions. However, preparing future drafts before verification is resolved introduces acceptance length and bonus token uncertainty. The drafter does not know how many draft tokens will be accepted, nor which continuation token will be determined by the verifier.

Diffusion Language Models Diffusion large language models (dLLMs) provide an alternative to autoregressive generation by denoising masked token sequences and predicting multiple positions in parallel. LLADA [25] demonstrates that this paradigm can scale to billion parameters language models and achieve competitive performance with strong autoregressive baselines. However, fully parallel dLLMs often suffer from fixed length generation [34], limited KV cache support [33, 23, 12], and many denoising steps [33, 31], which restrict their practical inference speed. Block diffusion models [3] alleviate these limitations by generating text block by block. Following this direction, recent work adapts autoregressive LLMs into block diffusion language models to improve parallelism, but typically requires substantial continued training and may introduce quality trade-offs [32, 7].

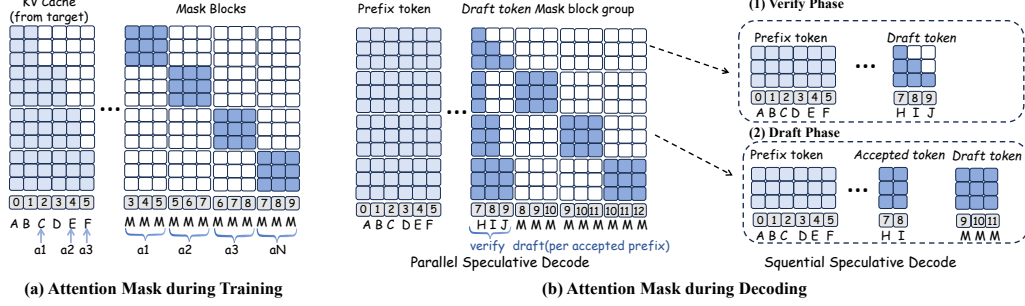


Figure 2: **Attention masks in FlexDraft.** (a) **Training.** The target performs causal forward to build the clean prefix KV cache. Mask tokens attend bidirectionally within each block and to the prefix, isolated from other blocks. (b) **Decoding.** Our method supports both parallel and sequential speculative decoding. In parallel mode, the latest draft is verified while candidates for all possible accepted lengths are prepared in the same forward pass. In sequential mode, drafting and verification are executed sequentially by selecting the verification row and its corresponding draft row from the parallel attention mask.

More recently, speculative decoding has also been incorporated into block diffusion to improve its performance in highly parallel generation settings. For example, hybrid diffusion and autoregressive methods such as TiDAR [21] combine diffusion style drafting with autoregressive sampling, but they lack the same target distribution guarantee as standard speculative decoding.

3 Preliminaries: Speculative Decoding

Speculative decoding accelerates autoregressive generation by decomposing each decoding step into drafting and verification. Let q and p_θ denote the drafter and target model distributions. The drafter first proposes k candidate tokens, $\hat{\mathbf{y}} = (\hat{y}_{n+1}, \dots, \hat{y}_{n+k})$, to approximate p_θ . The target model then verifies them in a single forward pass using *speculative sampling* [5], accepting draft tokens according to the ratio between p_θ and q and sampling from the residual target distribution at the first rejected position. We call this newly sampled token the *bonus token*. If r^{acc} draft tokens are accepted before the bonus token is produced, the committed length is $L^{\text{acc}} = r^{\text{acc}} + 1$.

Sequential speculative decoding. In the standard draft then verify schedule, a new draft begins only after verification completes. Given the accepted draft token count r^{acc} and the bonus token b , the drafter conditions on the complete verified context to generate the next candidate block:

$$\hat{\mathbf{y}}^{\text{next}} \sim q(\cdot \mid \mathbf{x}_{1:n}, \hat{\mathbf{y}}_{1:r^{\text{acc}}}, b). \quad (1)$$

Parallel speculative decoding. In contrast, parallel speculative decoding prepares future candidates before the current verification result is fully resolved. Since the accepted length L^{acc} and the bonus token b are unknown at draft time, the drafter cannot condition on the final verified context. Instead, it must prepare candidate drafts for multiple possible numbers of accepted draft tokens. Let $\mathcal{R} = \{0, 1, \dots, k\}$ denote the set of possible accepted draft token counts. An idealized formulation is to construct a candidate group indexed by $r \in \mathcal{R}$:

$$\mathcal{G} = \{\hat{\mathbf{y}}_r^{\text{next}} \sim q(\cdot \mid \mathbf{x}_{1:n}, \hat{\mathbf{y}}_{1:r})\}_{r \in \mathcal{R}}. \quad (2)$$

The case $r = 0$ corresponds to immediate rejection, where no draft token is accepted and $\hat{\mathbf{y}}_{1:0}$ is defined as an empty prefix. Each branch $\hat{\mathbf{y}}_r^{\text{next}}$ is generated under the hypothesis that the first r draft tokens are accepted and form the verified prefix for the next draft. Once verification completes, r^{acc} is determined and the corresponding branch $\hat{\mathbf{y}}_{r^{\text{acc}}}^{\text{next}}$ is selected from $\mathcal{G}$.

4 FlexDraft

FlexDraft is a lossless speculative decoding framework that combines lightweight block diffusion drafting, bonus-aware calibration, and batch-adaptive execution. It targets both the mutual waiting and memory access overhead of sequential speculative decoding at small batch sizes and the

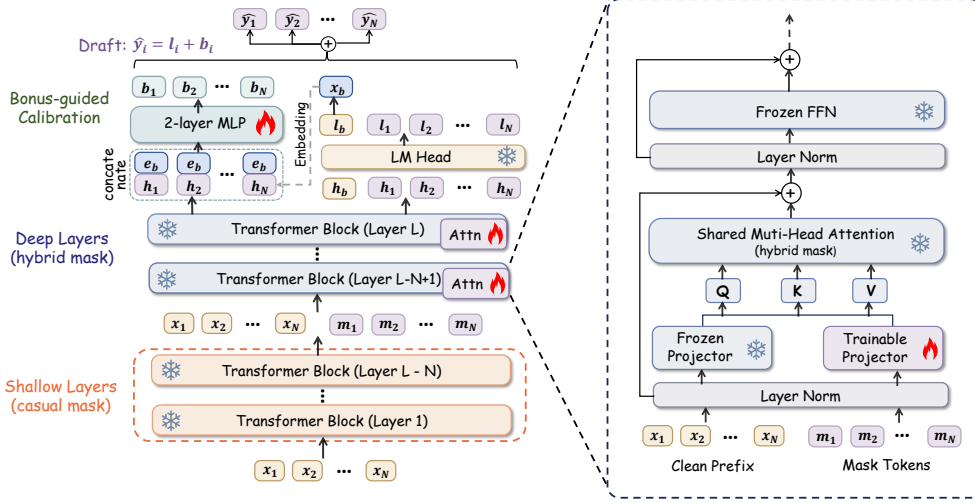


Figure 3: **Pipeline of FlexDraft.** Shallow layers process the clean prefix identically to a standard autoregressive forward pass. In the deep layers, mask tokens are appended to the prefix and routed through trainable attention projectors, enabling parallel draft prediction. Bonus-guided Calibration injects the verified bonus token embedding into a lightweight MLP to adjust draft logits, which improves draft quality.

redundant compute overhead of parallel speculative decoding at large batch sizes. Attn Tuning enables block diffusion drafting with only a small number of additional attention projector parameters while preserving the target model’s autoregressive distribution. Bonus-guided Calibration reduces the mismatch between bonus-free drafting and bonus-constrained verification, and Flex Decoding dynamically switches decoding modes and prunes verification length based on draft confidence to ensure robust acceleration across batch sizes. Together, these designs preserve the target distribution, improve draft quality, and prevent speedup collapse under large batch serving.

4.1 Attention Tuning

Our goal is to enable block diffusion drafting that produces k draft tokens in a single forward pass while preserving the target model’s autoregressive behavior, ensuring that our method remains lossless. Prior work on multi token prediction suggests that pretrained autoregressive models already encode information about future tokens in their intermediate representations, implying that full model continual pretraining may be unnecessary [27]. Motivated by this observation, we hypothesize that adapting only the attention projectors is sufficient to elicit parallel masked prediction, while keeping the FFN layers frozen allows the drafter to directly inherit the pretrained knowledge of the target model. In our design, mask tokens attend bidirectionally within each block and to the clean prefix, while remaining isolated from other blocks and invisible to clean prefix tokens, thereby preventing draft positions from interfering with prefix computation.

Dual attention projectors. Let $\mathbf{x}_{1:n}$ denote the clean prefix, comprising the prompt and all previously verified response tokens. As illustrated in Figure 3, FlexDraft processes the clean prefix identically to the original autoregressive model in the first $L-N$ layers, with no mask tokens involved. Starting from layer $L-N+1$, mask tokens $\mathbf{e}_m \in \mathbb{R}^d$ are appended to the hidden states of $\mathbf{x}_{1:n}$ as draft query positions. To separate drafting from standard autoregressive computation, we keep the original attention projectors $\mathbf{W}^l = (W_Q^l, W_K^l, W_V^l, W_O^l)$ frozen for all clean prefix tokens, and introduce trainable mask specific projectors $\tilde{\mathbf{W}}^l$ only for mask tokens in the final N layers. All other parameters, including the FFN layers and the original autoregressive projectors, remain frozen throughout training.

Training. To efficiently train multiple draft blocks in a single forward pass, we adopt a packed draft training strategy. We first process the clean sequence with the original target model under standard causal attention, obtaining reusable KV states for all clean positions. Each randomly sampled anchor position x_n defines the start of a draft block, and the subsequent k tokens $\{x_{n+1}, \dots, x_{n+k}\}$ serve as prediction targets. As illustrated in Figure 2, packed draft training is implemented by controlling visibility with attention masks and position IDs, rather than physically appending mask embeddings

after each anchor. For each anchor x_n , its k mask tokens are assigned position IDs $n+1$ through $n+k$, attend to the cached clean prefix up to x_n , and interact bidirectionally within the same block. Mask tokens from different draft blocks are mutually invisible, allowing multiple draft blocks to be packed into one forward pass without information leakage. These mask tokens are processed by the trainable mask specific attention projectors together with the frozen FFN layers inherited from the target model. This allows the drafter to reuse the target model’s pretrained knowledge while adapting only a small set of parameters. The resulting training process faithfully simulates block diffusion drafting at inference time and enables all k draft tokens to be predicted in parallel by the drafter q_ϕ . Following DFlash, we apply exponentially decaying weights to the block level cross entropy, emphasizing earlier draft positions because an early mismatch invalidates all subsequent draft tokens during verification. For a draft block starting at anchor x_n , the training objective is:

$$\mathcal{L}_{\text{draft}} = -\frac{1}{\sum_{i=1}^k w_i} \sum_{i=1}^k w_i \log q_\phi(x_{n+i} \mid \mathbf{x}_{1:n}), \quad w_i = \exp(-\lambda(i-1)), \quad (3)$$

where λ is a decay hyperparameter, set to 7 unless otherwise specified. The final training loss is averaged over all sampled anchors in the packed batch.

Inference. At each decoding step, FlexDraft executes drafting and verification concurrently. The previous draft $\hat{\mathbf{y}}^{(t)}$ is fed into the model, and the first $L - N$ layers process the clean prefix $\mathbf{x}_{1:n}$ autoregressively using the frozen projectors $\mathbf{W}^l$, identical to a standard forward pass, producing a KV cache that can be reused across all decoding steps. Starting from layer $L - N + 1$, k mask embeddings $\mathbf{e}_m^k$ are injected and routed through the trainable mask specific projectors $\tilde{\mathbf{W}}^l$, while clean prefix tokens continue through the frozen projectors. Using the hybrid attention mask, mask tokens attend bidirectionally within their block and to the clean prefix, and multiple candidate drafts are generated in parallel for all possible hypothetical acceptance lengths $\ell \in \mathcal{L}$. Once the actual accepted length L^{acc} is determined, the corresponding candidate $\hat{\mathbf{y}}_{L^{\text{acc}}}^{\text{next}}$ is selected for the next step.

4.2 Bonus-guided Calibration

In the parallel draft and verify paradigm, drafting proceeds without the bonus token, while verification is constrained by the resolved bonus token, which causes a draft verification mismatch. We introduce Bonus-guided Calibration, a lightweight module applied after the LM head, to bridge this mismatch by calibrating the draft logits with the bonus token.

Let $h_i \in \mathbb{R}^d$ denote the hidden state of mask position i at the final draft layer, and $e_b \in \mathbb{R}^d$ the embedding of the determined bonus token $b^{(t)}$. The calibration bias is jointly determined by both the bonus token and the draft content at each position. Thus, Bonus-guided Calibration concatenates e_b with h_i and passes the result through a lightweight two layer MLP to produce an additive bias over the vocabulary logits:

$$\tilde{\ell}_i = \ell_i + \text{MLP}(\text{Concat}(\mathbf{e}_b, \mathbf{h}_i)), \quad (4)$$

where $\ell_i \in \mathbb{R}^{|\mathcal{V}|}$ are the original draft logits at position i produced by the language modeling head. The calibrated logits $\tilde{\ell}_i$ are used to produce the final draft tokens, steering the draft distribution toward continuations consistent with $b^{(t)}$. During training, we use the ground truth token embedding as a proxy for e_b , and train the MLP jointly with Attn Tuning under an additional cross entropy loss, adding negligible parameter overhead.

4.3 Flex Decoding

Flex Decoding adapts the execution mode to the dominant bottleneck at different batch sizes, enabling a single trained drafter to support both inference modes without retraining. At small batch sizes, decoding is largely memory bound, so parallel draft and verify is preferable for avoiding mutual waiting bubbles and intermediate state exchange overhead. At large batch sizes, the target forward pass becomes compute bound, and the redundant branches of parallel speculative decoding make sequential draft then verify more efficient. Unlike prior parallel methods that rely on full model drafting, our drafter is localized to the final few layers through mask specific attention projectors and is therefore detachable. It can be inserted into the target forward pass for parallel execution or decoupled as a lightweight sequential draft step that reuses the clean prefix KV cache from target verification. Accordingly, Flex Decoding uses a batch size threshold to switch between the two

modes, adopting *Selective Verification* at small batch sizes to prune low confidence candidate lengths and switching to *Decoupled Execution* at large batch sizes to generate only the branch conditioned on the resolved accepted prefix.

Selective verification (small batch). In the parallel mode, FlexDraft prepares candidate drafts for multiple possible accepted lengths. Although the accepted length is unknown before verification, draft confidence provides a useful proxy for estimating which candidate lengths are more likely to be selected. Intuitively, higher confidence draft tokens are more likely to be accepted, and under a simple independence approximation across positions, the probability that the accepted length reaches position i can be estimated by the cumulative confidence. Let $p_i = q_\phi(\hat{y}_i \mid \mathbf{x}_{1:n})$ denote the draft probability of the i -th drafted token. We approximate this likelihood as

$$P(L^{\text{acc}} \geq i) \approx \prod_{r=1}^i p_r. \quad (5)$$

In this way, FlexDraft converts complete accepted length uncertainty into a coarse confidence-based estimate. Once the cumulative confidence falls below a threshold, we prune the remaining low probability accepted length candidates and verify only a subset of high confidence prefixes. This reduces the number of candidate prefixes verified by the target model without changing the draft block length, so each retained branch still preserves the same k token drafting capacity.

Decoupled execution (large batch). At large batch sizes, the redundant branches of parallel speculative decoding dominate the cost. Thus, Flex Decoding switches to decoupled execution, as shown in the sequential speculative decoding mode of Figure 2. Once verification resolves the accepted length L^{acc} , the drafter generates only one block conditioned on the verified prefix. Because the clean prefix computation is identical to the target autoregressive computation, the KV cache produced during target verification can be reused for drafting. Thus, only one branch is computed, avoiding redundant accepted length branches and preventing large batch speedup collapse.

5 Experiments

5.1 Experimental Setup.

We evaluate FlexDraft on Qwen3 series target models across code generation, mathematical reasoning, and general language understanding benchmarks, including HumanEval, MBPP, GSM8K, MATH, and MT-Bench. All experiments are conducted on NVIDIA A100 GPUs. We compare against representative speculative decoding baselines, including DFlash [6], EAGLE-3 [17], DART [20], BiTA [19] and Apple MTP [27]. We do not include TiDAR as a main baseline because it is a hybrid diffusion autoregressive method that does not generally provide the same lossless target distribution guarantee as standard speculative decoding, and its implementation has not been publicly released. Unless otherwise specified, all experiments are conducted with batch size 1, and all trainable methods are trained on the same 300K samples from `mlabonne/open-perfectblend`¹.

5.2 Main Results

Comparison under the same data. For a fair comparison, all trainable methods are trained on the same data and evaluated under identical settings. We first compare with parallel speculative decoding methods, including BiTA and Apple MTP. Their performance is constrained by the inherent limitations of the parallel speculative decoding paradigm, as well as limited trainable capacity and low acceptance rates. By explicitly mitigating bonus token uncertainty and reducing redundant computation, FlexDraft substantially outperforms existing parallel speculative decoding methods.

FlexDraft also consistently surpasses strong speculative decoding baselines, including EAGLE-3 and DFlash, across model scales and benchmarks. The key advantage is that FlexDraft reuses the target model’s frozen FFN parameters and only introduces lightweight mask specific attention projectors for draft tokens. As a result, increasing the draft depth incurs only small parameter overhead while producing longer accepted continuations. This enables FlexDraft to achieve higher acceptance lengths and speedups, while retaining the parallel drafting benefit that removes mutual waiting bubbles at small batch sizes.

¹<https://huggingface.co/datasets/mlabonne/open-perfectblend>

Table 1: Decoding performance on Qwen3 models. τ denotes average acceptance length. The highest speedup for each benchmark is highlighted in **bold**.

Model	Method	GSM8K		MATH		HumanEval		MBPP		MT-Bench	
		τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup
Qwen3-8B	BiTA	1.68	1.34×	1.63	1.30×	1.49	1.21×	1.47	1.19×	1.47	1.15×
	Apple MTP	3.30	2.77×	3.14	2.61×	2.49	2.09×	2.44	2.04×	2.11	1.63×
	EAGLE-3	4.84	3.40×	4.88	3.34×	3.57	2.47×	3.48	2.45×	2.81	1.75×
	dFlash	4.87	3.47×	4.69	3.27×	3.10	2.28×	3.23	2.33×	2.53	1.64×
	FlexDraft	6.12	4.57×	5.98	4.40×	4.15	3.25×	3.93	3.04×	3.22	2.13×
Qwen3-4B	EAGLE-3	4.19	2.82×	4.03	2.82×	3.01	2.09×	2.98	2.08×	2.45	1.53×
	dFlash	5.18	3.66×	4.93	3.42×	3.41	2.47×	3.45	2.46×	2.78	1.76×
	FlexDraft	5.28	4.04×	5.35	4.02×	3.73	2.93×	3.56	2.82×	3.01	1.99×
Qwen3-1.7B	EAGLE-3	3.31	2.12×	3.36	2.14×	2.39	1.49×	2.41	1.51×	1.96	1.31×
	dFlash	4.87	3.25×	4.58	3.01×	3.20	2.20×	3.17	2.16×	2.72	1.68×
	FlexDraft	4.98	3.67×	4.94	3.61×	3.41	2.61×	3.26	2.48×	3.03	1.99×

Table 2: Decoding performance on Qwen3-8B. τ denotes average acceptance length. The highest speedup for each benchmark is highlighted in **bold**.

Model	Method	GSM8K		MATH		HumanEval		MBPP		MT-Bench	
		τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup
Qwen3-8B	Apple MTP	3.30	2.77×	3.14	2.61×	2.49	2.09×	2.44	2.04×	2.11	1.63×
	DART	2.71	2.28×	2.70	2.29×	2.95	2.52×	2.98	2.39×	3.03	2.27×
	EAGLE-3	3.80	2.57×	3.61	2.44×	3.74	2.52×	3.31	2.24×	3.04	1.89×
	dFlash	6.41	4.55×	7.93	5.50×	6.47	4.68×	5.93	4.16×	4.29	2.53×
	FlexDraft	7.98	5.88×	8.03	5.79×	5.55	4.26×	6.16	4.46×	4.16	2.58×

Comparison under scaled training. Table 2 further compares FlexDraft with released or publicly reported SOTA methods on Qwen3-8B under scaled training, evaluating its scaling potential in a more competitive setting. FlexDraft consistently outperforms DART and EAGLE-3 across most benchmarks. Compared with DFlash, FlexDraft achieves comparable or slightly better performance, despite not having access to the training data used by DFlash. These results demonstrate that FlexDraft scales effectively and remains competitive with strong publicly reported methods under a non-data-matched comparison.

Batch size study. Figure 4 evaluates the performance under different batch sizes. Seq. SD and Par. SD are two inference modes derived from the same trained FlexDraft model, rather than separately trained variants. At small batch sizes, parallel speculative decoding achieves higher speedup because drafting and verification can be overlapped, effectively reducing the mutual waiting bubbles between the two stages. However, as the batch size increases, the redundant computation introduced by parallel speculative decoding becomes more pronounced, making sequential speculative decoding more favorable by drafting only along the resolved accepted branch. Based on this observation, we use a batch size of 2 as the threshold for switching from parallel to sequential speculative decoding in FlexDraft. This adaptive strategy allows FlexDraft to benefit from parallel execution in small batch regimes while avoiding excessive overhead in large batch regimes. As a result, FlexDraft consistently achieves higher speedup than EAGLE-3 and dFlash across batch sizes, demonstrating both the effectiveness of our training design and the necessity of batch-aware inference selection.

5.3 Ablation

Latency of a single draft and verify step. Figure 5 compares the latency of one draft and verify step. EAGLE and dFlash first generate draft tokens and then verify them with the target model, so their latency accumulates across the two stages. In contrast, FlexDraft avoids a separate autoregressive drafting pass. It generates a block of draft tokens in one pass while target verification is performed, thereby avoiding the sequential draft then verify pipeline and reducing per step latency.

Effect of the number layer of draft model. Figure 6 studies the number of draft layers. A larger draft depth generally improves acceptance length by strengthening draft target alignment. While 13 layers yields slightly higher speedup in the parallel setting, it increases trainable parameters and draft

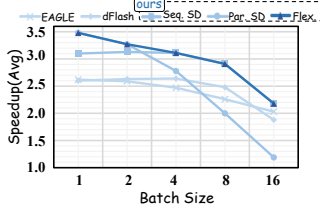


Figure 4: **Speedup across batch sizes.**

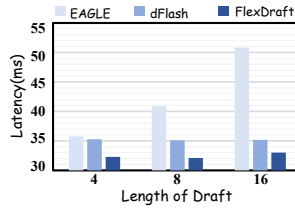


Figure 5: **Execution time of a single draft and verify step.**

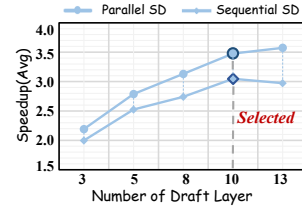


Figure 6: **Effect of the number of draft layers.**

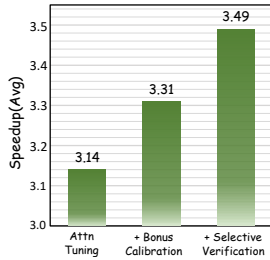


Figure 7: **Ablation analysis of speedup.**

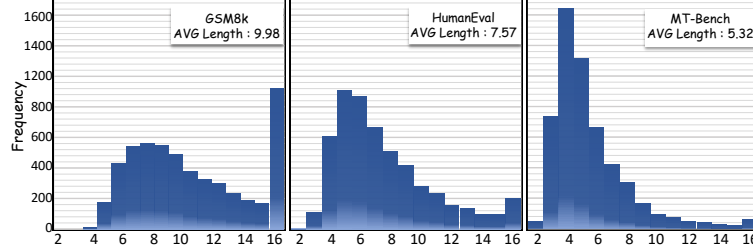


Figure 8: **Distribution of selective verification length.** The selected verification length varies across tasks.

computation, which becomes less favorable under decoupled execution. We therefore use 10 layers by default, as it provides a better trade-off between speedup and decoding overhead.

Effectiveness of components. Figure 7 shows the contribution of each component. Starting from Attn Tuning, adding Bonus-guided Calibration improves the average speedup by better aligning drafts with the bonus-constrained verification. Selective Verification further improves speedup by pruning low confidence verification positions and reducing redundant target forward computation. These results show that the two components are effective.

6 Discussion

Adaptive verification length across tasks. Figure 8 shows the distribution of selective verification length across tasks. The average is correlated with the average acceptance length, where tasks with longer accepted continuations receive more verification. This indicates that selective verification performs sample-wise dynamic verification instead of using a fixed verification length, which can exploit potential speedup while avoiding redundant target forward computation.

Reusing target knowledge for drafting. Attn Tuning only finetunes the attention projectors while reusing the target model’s frozen FFN, allowing the drafter to inherit pretrained knowledge from target model. To analyze this effect, we compare Attn Tuning with a fully finetuned drafter under the same trainable parameter budget. The full parameter drafter relies only on its trained parameters at inference time, whereas Attn Tuning can additionally leverage the frozen FFN of the target model. As shown in Table 3, this design consistently improves both acceptance length and speedup across benchmarks, demonstrating the benefit of reusing target knowledge for drafting.

Table 3: Effect of target knowledge reuse.

Benchmark	Setting	τ	Speedup
GSM8K	Full param	3.78	3.04
	Attn Tuning	5.86	4.23
HumanEval	Full param	2.57	2.10
	Attn Tuning	3.99	2.94
MT-Bench	Full param	2.14	1.60
	Attn Tuning	3.06	1.90

7 Conclusion

We introduced FlexDraft, a lossless speculative decoding framework that combines lightweight block diffusion drafting with adaptive execution. By tuning only attention projectors, calibrating drafts with the resolved bonus token, and dynamically switching speculative decoding strategies across

batch sizes, FlexDraft reduces both mutual waiting bubbles and redundant overhead. Experiments on Qwen3 models demonstrate consistent speedups over strong baselines across reasoning, coding, and chat benchmarks, while preserving the target model distribution. FlexDraft therefore offers a practical path toward efficient LLM inference under diverse serving conditions.

References

- [1] Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, and Emad Barsoum. Pard: Accelerating llm inference with low-cost parallel draft model adaptation. In *International Conference on Learning Representations*, 2026.
- [2] Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. Hydra: Sequentially-dependent draft heads for medusa decoding. *arXiv preprint arXiv:2402.05109*, 2024.
- [3] Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *International Conference on Learning Representations*, 2025.
- [4] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 5209–5235. PMLR, 2024.
- [5] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- [6] Jian Chen, Yesheng Liang, and Zhijian Liu. Dflash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026.
- [7] Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, and Bowen Zhou. SDAR: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*, 2025.
- [8] Jacob K. Christopher, Brian R. Bartoldson, Tal Ben-Nun, Michael Cardei, Bhavya Kailkhura, and Ferdinando Fioretto. Speculative diffusion decoding: Accelerating language generation through diffusion. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 12042–12059, Albuquerque, New Mexico, 2025. Association for Computational Linguistics.
- [9] DeepSeek-AI. Deepseek-v3 technical report, 2024.
- [10] Milan Gritta, Huiyin Xue, and Gerasimos Lampouras. DReSD: Dense retrieval for speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 19822–19832, Vienna, Austria, 2025. Association for Computational Linguistics.
- [11] Zhenyu He, Zexuan Zhong, Tianle Cai, Jason Lee, and Di He. REST: Retrieval-based speculative decoding. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1582–1595, Mexico City, Mexico, 2024. Association for Computational Linguistics.
- [12] Jianuo Huang, Yaojie Zhang, Yicun Yang, Benhao Huang, Biqing Qi, Dongrui Liu, and Linfeng Zhang. Mask tokens as prophet: Fine-grained cache eviction for efficient dllm inference. *arXiv preprint arXiv:2510.09309*, 2025.
- [13] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023.

- [14] Guanghao Li, Zhihui Fu, Min Fang, Qibin Zhao, Ming Tang, Chun Yuan, and Jun Wang. Diffuspec: Unlocking diffusion language models for speculative decoding. *arXiv preprint arXiv:2510.02358*, 2025.
- [15] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 7421–7432. Association for Computational Linguistics, 2024.
- [16] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 28935–28948. PMLR, 2024.
- [17] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Advances in Neural Information Processing Systems*, 2025.
- [18] Zeping Li, Xinlong Yang, Ziheng Gao, Ji Liu, Guanchen Li, Zhuang Liu, Dong Li, Jinzhang Peng, Lu Tian, and Emad Barsoum. Amphista: Bi-directional multi-head decoding for accelerating LLM inference. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8925–8938, Albuquerque, New Mexico, 2025. Association for Computational Linguistics.
- [19] Feng Lin, Hanling Yi, Yifan Yang, Hongbin Li, Xiaotian Yu, Guangming Lu, and Rong Xiao. Bitar: Bi-directional tuning for lossless acceleration in large language models. *Expert Systems with Applications*, 279:127305, 2025.
- [20] Fuliang Liu, Xue Li, Ketai Zhao, Yinxi Gao, Ziyang Zhou, Zhonghui Zhang, Zhibin Wang, Wanchun Dou, Sheng Zhong, and Chen Tian. Dart: Diffusion-inspired speculative decoding for fast llm inference. *arXiv preprint arXiv:2601.19278*, 2026.
- [21] Jingyu Liu, Xin Dong, Zhifan Ye, Rishabh Mehta, Yonggan Fu, Vartika Singh, Jan Kautz, Ce Zhang, and Pavlo Molchanov. Tidar: Think in diffusion, talk in autoregression. *arXiv preprint arXiv:2511.08923*, 2025.
- [22] Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, Winston Hu, and Xiao Sun. Pearl: Parallel speculative decoding with adaptive draft length. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [23] Zhiyuan Liu, Yicun Yang, Yaojie Zhang, Junjie Chen, Chang Zou, Qingyuan Wei, Shaobo Wang, and Linfeng Zhang. dlm-cache: Accelerating diffusion large language models with adaptive caching. *arXiv preprint arXiv:2506.06295*, 2025.
- [24] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 932–949. ACM, 2024.
- [25] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In *Advances in Neural Information Processing Systems*, 2025.
- [26] Guofeng Quan, Wenfeng Feng, Chuzhan Hao, Guochao Jiang, Yuewei Zhang, and Hao Henry Wang. RASD: Retrieval-augmented speculative decoding. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 6167–6177, Vienna, Austria, July 2025. Association for Computational Linguistics.

- [27] Mohammad Samragh, Arnav Kundu, David Harrison, Kumari Nishu, Devang Naik, Minsik Cho, and Mehrdad Farajtabar. Your llm knows the future: Uncovering its multi-token prediction potential. *arXiv preprint arXiv:2507.11851*, 2025.
- [28] Yuhao Shen, Junyi Shen, Quan Kong, Tianyu Liu, Yao Lu, and Cong Wang. Specbranch: Speculative decoding via hybrid drafting and rollback-aware branch parallelism. In *International Conference on Learning Representations*, 2026.
- [29] Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.
- [30] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- [31] Qingyan Wei, Yaojie Zhang, Zhiyuan Liu, Puyu Zeng, Yuxuan Wang, Biqing Qi, Dongrui Liu, and Linfeng Zhang. Accelerating diffusion large language models with slowfast sampling: The three golden principles. *arXiv preprint arXiv:2506.10848*, 2025.
- [32] Chengyue Wu, Hao Zhang, Shuchen Xue, Shizhe Diao, Yonggan Fu, Zhijian Liu, Pavlo Molchanov, Ping Luo, Song Han, and Enze Xie. Fast-dllm v2: Efficient block-diffusion llm. *arXiv preprint arXiv:2509.26328*, 2025.
- [33] Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025.
- [34] Zirui Wu, Lin Zheng, Zihui Xie, Jiacheng Ye, Jiahui Gao, Shansan Gong, Yansong Feng, Zhenguo Li, Wei Bi, Guorui Zhou, and Lingpeng Kong. Dreamon: Diffusion language models for code infilling beyond fixed-size canvas. In *International Conference on Learning Representations*, 2026.
- [35] Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 7655–7671, Bangkok, Thailand, 2024. Association for Computational Linguistics.
- [36] Kaiqi Zhang, Jing Zhao, and Rui Chen. KOALA: Enhancing speculative decoding for LLM via multi-layer draft heads with adversarial learning. *arXiv preprint arXiv:2408.08146*, 2024.
- [37] Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Ros-tamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. Distillspec: Improving speculative decoding via knowledge distillation. In *The Twelfth International Conference on Learning Representations*, 2024.

A Appendix

A.1 Robustness to sampling temperature.

Table 4: Decoding performance on Qwen3-8B under different temperatures ($T = 0.3, 0.6, 1.0$). τ denotes average acceptance length. The highest speedup for each benchmark is highlighted in **bold**.

Temp	Method	GSM8K		MATH		HumanEval		MBPP		MT-Bench	
		τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup
$T = 0.3$	dFlash	4.83	3.44×	4.64	3.24×	3.09	2.26×	3.21	2.32×	2.51	1.62×
	EAGLE-3	4.67	3.27×	4.61	3.21×	3.58	2.42×	3.53	2.41×	2.44	1.74×
	FlexDraft	5.61	4.20×	5.45	3.99×	3.85	2.99×	3.58	2.75×	2.92	1.92×
$T = 0.6$	dFlash	4.63	3.34×	4.48	3.12×	3.01	2.21×	3.14	2.28×	2.50	1.60×
	EAGLE-3	4.60	3.21×	4.36	3.11×	3.44	2.36×	3.47	2.36×	2.37	1.70×
	FlexDraft	5.11	3.83×	4.91	3.56×	3.46	2.70×	3.34	2.60×	2.66	1.81×
$T = 1.0$	dFlash	4.46	3.18×	4.23	2.93×	2.92	2.14×	3.06	2.20×	2.36	1.56×
	EAGLE-3	4.41	3.12×	3.96	2.88×	3.24	2.26×	3.21	2.27×	2.27	1.67×
	FlexDraft	4.61	3.43×	4.61	3.44×	4.33	3.09×	2.99	2.29×	2.41	1.65×

We further evaluate robustness under different sampling temperatures, i.e., $T = 0.3, 0.6, 1.0$. Table 4 shows that FlexDraft remains competitive across all settings and achieves the best speedup in most cases, indicating stable performance of our method.

A.2 Limitations

This work focuses on improving the efficiency of speculative decoding for large language model inference. A main limitation is that the observed speedup is inherently hardware- and workload-dependent. Although FlexDraft is designed to reduce mutual waiting between drafting and verification, the actual acceleration depends on factors such as GPU architecture, memory bandwidth, batch size, sequence length, and the relative computational intensity of the draft and verification stages. For example, at small batch sizes, decoding is often more memory-bound and the benefit of overlapping drafting with verification can be more pronounced. As batch size increases, the target forward pass may become more compute-bound and dominate end-to-end latency, reducing the relative gain from parallel execution.

Therefore, the measured speedup of FlexDraft may vary across hardware platforms, model scales, prompt distributions, and serving configurations. While our experiments cover representative models, benchmarks, and batch size settings, future work could further evaluate the method under more diverse deployment environments, including heterogeneous hardware, longer context workloads, and real multi user serving traces.

A.3 Broader Impacts

This work aims to improve the efficiency of large language model inference. Potential positive impacts include reducing inference latency, serving cost, and energy consumption, which may make LLM-based applications more accessible to researchers, developers, and users with limited computational resources. More efficient inference can also improve the practicality of interactive applications where response latency is critical.

At the same time, improving LLM inference efficiency may also lower the cost of deploying harmful or undesirable generative applications, such as spam generation, misinformation, automated harassment, or other misuse of language models. Our work does not introduce new generative capabilities, collect private data, or release a new pretrained model. Nevertheless, because the proposed method can accelerate existing LLMs, it should be used together with appropriate model-level safety mechanisms, deployment policies, monitoring systems, and usage restrictions determined by the model provider or application developer.

A.4 Existing Assets and Licenses

We use existing models, datasets, and codebases only for research and evaluation purposes. All existing assets are cited in the main paper, and their licenses or official terms of use are summarized in Table 5. We do not redistribute third-party model weights or datasets beyond their original licenses or official access channels.

Table 5: Existing assets used in this work.

Asset	Type	Usage	License / Terms of Use
Qwen3 series models	Model	Target LLMs	Apache-2.0
open-perfectblend	Dataset	Draft-model training	Apache-2.0 / MIT mixture
HumanEval	Benchmark	Code evaluation	MIT License
MBPP	Benchmark	Code evaluation	CC BY 4.0 / Apache-2.0
GSM8K	Benchmark	Math evaluation	MIT License
MATH	Benchmark	Math evaluation	MIT License
MT-Bench / FastChat	Benchmark / code	Chat evaluation	Apache-2.0
EAGLE-3	Code / baseline	Baseline comparison	Apache-2.0
DFlash	Code / baseline	Baseline comparison	MIT License
DART	Code / baseline	Baseline comparison	Apache-2.0
Hugging Face Transformers	Codebase	Model inference	Apache-2.0
PyTorch	Codebase	Training / inference	BSD-style License
NCCL	Library	Multi-GPU backend	Apache-2.0 / BSD components

Before publication, we will verify the license and usage terms of each asset against its official source and ensure that the released code, scripts, and checkpoints comply with the corresponding requirements.

A.5 Reproducibility Details

We provide the main experimental details required to reproduce the results reported in this paper. All experiments are conducted on Qwen3 series target models and evaluated on five benchmarks covering code generation, mathematical reasoning, and general language understanding: HumanEval, MBPP, GSM8K, MATH, and MT-Bench. The compared baselines include DFlash, EAGLE-3, DART, BiTA, and Apple MTP. Unless otherwise stated, all methods are evaluated under the same target model, tokenizer, benchmark split, prompt format, hardware platform, and inference configuration.

For a fair comparison under the same data setting, all trainable draft models are trained on 300K samples from mlabonne/open-perfectblend, following the training data scale used by EAGLE-3. We use the same training data for our method and the reproduced baselines whenever the corresponding training pipeline is available. The default draft configuration uses 10 draft layers and a block size of 16. Draft models are trained for 6 epochs using AdamW with a learning rate of 6×10^{-4} , gradient clipping threshold 1.0, and a cosine learning rate schedule with a warmup ratio of 0.04. For each input sequence, 128 anchor positions are randomly sampled as training targets. MLP hidden state size=256

All inference experiments are conducted on NVIDIA A100 GPUs. We report speedup, accepted length, and task performance under identical hardware and inference settings. For robustness evaluation, we test different sampling temperatures, including $T = 0.3$, $T = 0.6$, and $T = 1.0$. For batch size analysis, we evaluate the decoding performance across different batch sizes to compare the behavior of parallel draft and verify execution and sequential draft then verify execution. All evaluations use the SDPA attention implementation, with the maximum generation length set to 1024.

For latency and throughput measurement, we use the same input prompts, maximum generation length, batch size, and sampling configuration for all compared methods. End-to-end decoding time includes both drafting and target model verification. Speedup is computed relative to the autoregressive decoding baseline under the same model, hardware, and generation setting. For ablation studies, we evaluate the contribution of Attn Tuning, Bonus-guided Calibration, and Selective Verification by enabling the components incrementally while keeping all other configurations unchanged.

A.6 Experiments Compute Resources

All experiments are conducted on a server equipped with 8 NVIDIA A100 GPUs with 80GB memory per GPU. The server has 120 CPU cores and 1800 GiB of system memory. For training, FlexDraft takes approximately 8 hours on 8 A100 GPUs, corresponding to about 64 A100 GPU-hours. For inference evaluation, the full set of decoding experiments takes approximately 1 hour on 8 A100 GPUs, corresponding to about 8 A100 GPU-hours. We use Python 3.11, PyTorch 2.9.0, Transformers 4.57.3, Datasets 4.4.1, and Accelerate 1.11.0 for all experiments.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The main claims in the abstract and introduction are supported by experimental results presented in Sections 5. We explicitly state the scope of the method, including the evaluated models, datasets, and serving settings.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: We discuss the limitations of our approach in Appendix A.2, focusing on the hardware- and workload-dependent nature of inference speedup, including the effects of GPU architecture, memory bandwidth, batch size, sequence length, and draft–verification computational balance.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include formal theoretical results; its main contributions are algorithmic and empirical.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We provide sufficient details to reproduce the main experiments, including model checkpoints, datasets, decoding settings, batch sizes, evaluation metrics, and hardware configurations in Appendix A.5.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in

some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide the code and model checkpoints in the supplementary material to support reproducibility of the main results. We also plan to publicly release the evaluation code and checkpoints upon acceptance.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The paper specifies the evaluated models, datasets, decoding parameters, batch sizes, hardware settings, and baseline configurations in Appendix A.5.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: We do not report error bars for all experiments because each run requires substantial GPU resources. To improve reliability, we report consistent trends across multiple models, datasets, and batch-size settings.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We report the compute resources used for each major experiment, including GPU type, number of GPUs, CPU cores, system memory, software environment, and approximate runtime in Appendix A.6.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research conforms to the NeurIPS Code of Ethics. We use publicly available models and datasets, preserve submission anonymity, and do not involve human subjects or private user data.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: We discuss potential positive impacts, including reduced inference latency, serving cost, and energy consumption, as well as potential negative impacts, such as lowering the cost of deploying harmful generative applications, in Appendix A.3.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release new pretrained language models, image generators, scraped datasets, or other high-risk assets. The released materials are limited to inference algorithms and experimental code.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We properly cite the existing models, datasets, and codebases used in our experiments, and report their licenses and terms of use where available in Appendix A.4.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: We release anonymized code with documentation, environment setup instructions, and scripts for reproducing the main experiments in the supplemental material.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing experiments or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve human subjects, so IRB approval or equivalent review is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are central to the experimental evaluation of this work. We describe the target and draft models, their roles in the proposed decoding framework, and the corresponding evaluation settings in Sections 5.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.